

Topic Classification Report

Abstract

This project presents a topic classification system that predicts the category of a given text document. The solution combines traditional machine learning models with a custom deep learning architecture built from scratch. The implementation includes data preprocessing, custom tokenization, vocabulary creation, feature engineering, model training, evaluation, and inference. No pretrained language models such as BERT or GPT were used.

1. Introduction

Topic classification is one of the most important tasks in Natural Language Processing (NLP). It automatically assigns a category or topic to a text document.

Applications include:

News categorization

Email classification

Customer support ticket routing

Social media monitoring

Content recommendation

The objective of this project is to build a scalable topic classification system capable of processing millions of text samples.

2. Problem Statement

Given a dataset containing:

Column	Description
--------	-------------

DATA	Input text
------	------------

TOPIC	Target class
-------	--------------

Develop a model that predicts the correct topic for unseen text.

Constraints:

No pretrained language models

Build custom tokenizer

Build custom vocabulary

Train custom deep learning model

Evaluate using standard classification metrics

3. Dataset

Dataset Information

Property	Value
----------	-------

Format	Parquet
--------	---------

Size	4.28 GB
------	---------

Records	10 Million
---------	------------

Columns	DATA, TOPIC
---------	-------------

4. Data Preprocessing

The following preprocessing steps were applied:

Lowercase conversion

URL removal

Email removal

HTML tag removal

Number removal

Special character removal

Extra whitespace removal

Duplicate removal

Missing value removal

5. Feature Engineering

TF-IDF

Configuration:

Max Features: 200000

N-Gram Range: (1,2)

Min Document Frequency: 5

Max Document Frequency: 0.95

Advantages:

Fast

Memory efficient

Good baseline

Custom Vocabulary

Special Tokens:

<PAD>

<UNK>

Vocabulary built from training data only.

6. Model Architecture

Input Text

|

Tokenizer

|

Vocabulary Encoding

|

Embedding Layer

|

BiLSTM

|

Attention Layer

|

Dropout

|

Fully Connected Layer

|

Softmax

|

Predicted Topic

7. Baseline Models

The following traditional machine learning models were implemented for comparison:

Multinomial Naive Bayes

Advantages:

Fast training

Low memory usage

Limitations:

Limited contextual understanding

Logistic Regression

Advantages:

Better generalization

Strong baseline

Linear SVM

Advantages:

High accuracy on sparse TF-IDF features

8. Deep Learning Model

The custom deep learning model consists of:

Embedding Layer

Bidirectional LSTM

Attention Mechanism

Dropout

Fully Connected Output Layer

No pretrained embeddings or pretrained transformers were used.

9. Training Strategy

Train/Validation Split: 80/20

Optimizer: Adam

Mixed Precision Training (if used)

Learning Rate Scheduler

Early Stopping

Checkpoint Saving

Experimental Results

Replace the values below with the actual metrics from your experiments.

10. Evaluation Metrics

The following metrics were used:

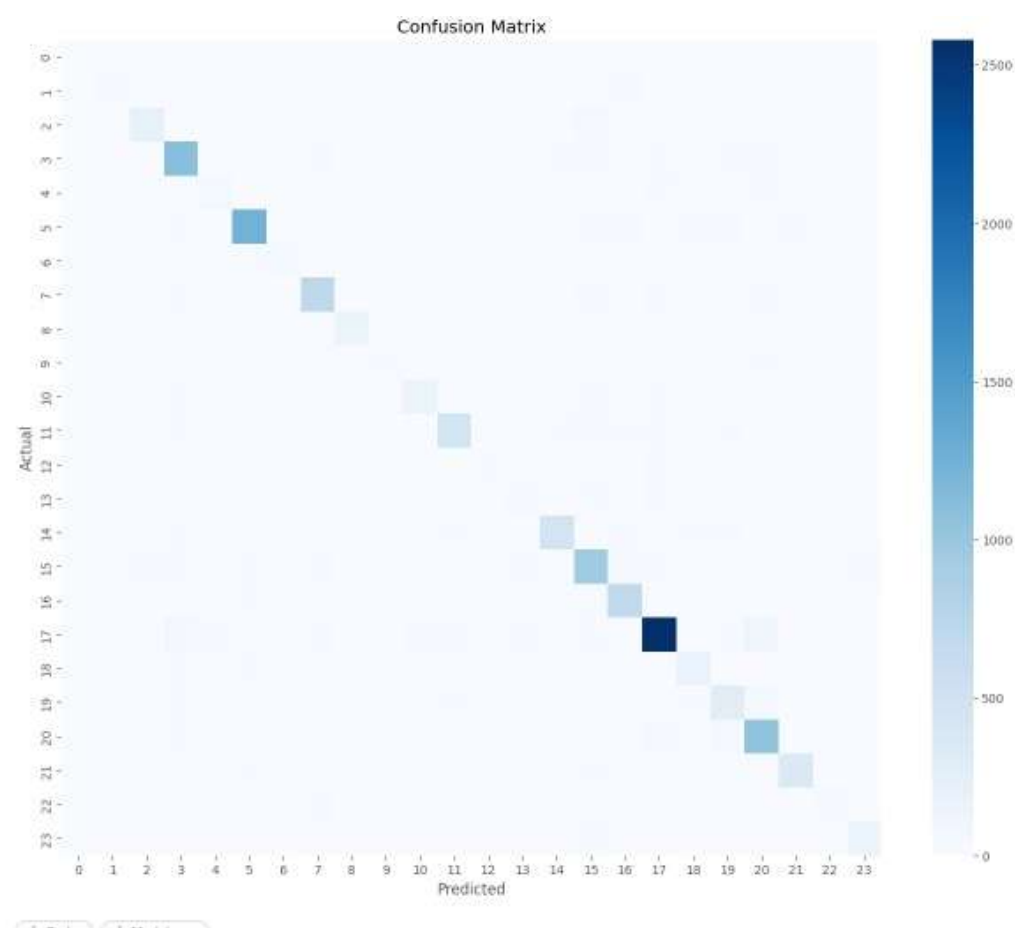
Accuracy

Precision

Recall

F1 Score

Confusion Matrix



Topic Classification Report

	Model	Accuracy	Precision	Recall	F1 Score
2	Linear SVM	0.860169	0.858127	0.860169	0.856547
1	Logistic Regression	0.835031	0.836276	0.835031	0.824635
0	Naive Bayes	0.778893	0.795067	0.778893	0.771976

11.Error Analysis

Common sources of error:

Semantically similar topics

Short text samples

Class imbalance

Ambiguous language

Possible improvements:

Increase training data for minority classes

Hyperparameter tuning

Larger embedding dimensions

Better attention mechanism

12. Challenges Faced

Large dataset (10M rows)

Memory management

Long training time

Vocabulary size optimization

Class imbalance

13.Conclusion

This project successfully demonstrates an end-to-end topic classification pipeline using both traditional machine learning and a custom deep learning model. The implementation avoids pretrained language models and instead builds every stage—from tokenization to classification—from scratch. The modular architecture allows future enhancements while providing a scalable solution for large text datasets.